Marathwada Mitra Mandal's
COLLEGE OF ENGINEERING, Karvenagar, Pune - 52
An Autonomous Institute affiliated to SPPU
NAAC: A ++ | NBA Accredited **Best College Award 2019 SPPU**
website: www.mmcoe.edu.in email id: mmcoe@mmcoe.edu.in

A Project Report
On

College Seminar Hall Reservation System (SmartSem)

By

Ayush Shedge – B24CE1108

Sanskar Tathe - B24CE1109

Aarya Bhave - B24CE1110

Under the guidance of

Reshma Kapadi mam

Department of Computer Engineering

Marathwada Mitra Mandal's College of Engineering

Class: TY-I, Sem-I

Academic Year 2026-27



Marathwada Mitra Mandal's
COLLEGE OF ENGINEERING, Karvenagar, Pune - 52
An Autonomous Institute affiliated to SPPU
NAAC: A ++ | NBA Accredited **Best College Award 2019 SPPU**
website: www.mmcoe.edu.in email id: mmcoe@mmcoe.edu.in

22-10-2026

CERTIFICATE

This is to certify that Students Name(prn) of class TY Computer Engineering have successfully completed their project work on **"College Seminar Hall Reservation System (SmartSem)"** at MARATHWADA MITRA MANDAL'S COLLEGE OF ENGINEERING in the partial fulfillment of the Graduate Degree course in T.E at the Department of Computer Engineering in the academic Year 2026-2027 Semester –I.

Dr. Neha Jain
Shiravale
Guide

Dr. S.S

Head of the Department
Computer Engineering

Acknowledgement

I feel great pleasure in expressing my deepest sense of gratitude and sincere thanks to my guide **Dr. Neha Jain** for their valuable guidance during the Project work, without which it would have been a very difficult task. I have no words to express my sincere thanks for valuable guidance, extreme assistance and cooperation extended to all the **Staff Members** of my Department.

This acknowledgement would be incomplete without expressing my special thanks to **Dr. S.S Shiravale**, Head of the Department (Computer Engineering) for their support during the work.

I would also like to extend my heartfelt gratitude to my **Principal, Dr. Sachin Sakhare** who provided a lot of valuable support, mostly being behind the veils of college bureaucracy.

Last but not least I would like to thank all the Teaching, Non- Teaching staff members of my Department, my parents and my colleagues who helped me directly or indirectly for completing this Project successfully.

Students Name(PRN)

ABSTRACT

Seminar halls in a college are a shared and limited resource that several departments and student groups request at the same time. When seats are handed out purely on a first-come basis, groups may end up holding partial bookings while waiting for the remaining seats, and then no group is able to complete its event. This project, SmartSem – College Seminar Hall Reservation System, applies the Banker's Algorithm, a deadlock-avoidance technique from Operating Systems, to the seat reservation problem. Seats are treated as identical resource instances and each student group as a process that declares a maximum seat requirement in advance. Every booking request is tested by temporarily allocating the seats and running the safety algorithm; the request is approved only if a safe sequence exists, otherwise it is placed in a FIFO waiting list. When a group cancels seats, the released seats are used to re-examine the waiting list automatically. The system is implemented as a menu-driven console application in Java using only the java.util package, and it offers booking, cancellation, a safe/unsafe dashboard and a waiting-list view. It was tested with 19 test cases covering normal, boundary and invalid inputs; 18 behaved as expected and the one failure, non-numeric input, is documented as a limitation. The results show that the system never enters an unsafe state and resolves waiting requests correctly after cancellations, demonstrating a practical use of deadlock avoidance in a campus setting.

TABLE OF CONTENTS

Chapter 1: Introduction

1.1 Project Overview

1.2 Problem Definition

1.3 Motivation

1.4 Objectives

1.5 Scope of the Project

Chapter 2: Theoretical Background

2.1 Selected Concept

2.2 Relevant Terminology

2.3 Algorithm(s) / Technique(s)

2.4 Technologies and Tools Used

Chapter 3: System Analysis and Design

3.1 Functional Requirements

3.2 Hardware and Software Requirements

3.3 System Architecture / Block Diagram/Flowchart

3.5 Algorithm / Pseudocode

Chapter 4: Implementation

4.1 Implementation Approach

4.2 Module Description

4.3 Important Source Code with Explanation

4.4 Execution Procedure

4.5 Input and Output Screenshots

Chapter 5: Testing and Results

5.1 Test Cases

5.2 Actual Results

5.3 Result Analysis and Observations

5.4 Limitations

Conclusion

Future Scope

References

MARATHWADA MITRAMANDAL, PUNE
'Welfare of Masses'
Est.1967

CHAPTER 1: INTRODUCTION

1.1 Project Overview

SmartSem – College Seminar Hall Reservation System is a menu-driven console application written in Java that manages the reservation of seats in a college seminar hall. Several departments or student groups (for example Computer, IT and ENTC) declare in advance the maximum number of seats they may need for their events. They then book seats in parts, cancel seats when plans change, and check the status of the hall through a dashboard.

The SPOS concept addressed is **deadlock avoidance using the Banker's Algorithm**. The seminar hall plays the role of the operating system, the seats are identical resource instances, and each student group is a process with a declared maximum claim. Every booking is treated as a resource request. Before a request is granted, the system temporarily allocates the seats and runs the safety algorithm; the seats are actually given to the group only if the resulting state is safe. Requests that are not safe, or that ask for more seats than are currently free, are kept in a First-In-First-Out (FIFO) waiting list and are re-examined whenever seats are released.

The application provides the following operations:

- Set up the hall capacity and the participating groups with their maximum seat requirements.
- Book seats for a group, with the Banker's safety check applied to each request.
- Cancel (release) seats, followed by an automatic scan of the waiting list.
- View the dashboard showing total, available, allocated, maximum and need values, the system status (SAFE/UNSAFE) and a safe sequence.
- View the FIFO waiting list of pending requests.

1.2 Problem Definition

A seminar hall has a fixed number of seats, and many groups want to reserve seats for their events. Each group knows the largest number of seats it will need, but it asks for them in several instalments. If the seats are simply given out in the order in which requests arrive, a situation can arise in which every group holds some seats, no seats are left, and every group is waiting for more seats before it can hold its event. Because no group can finish, no group releases its seats, and the system is stuck. This is the same hold-and-wait situation that causes deadlock in an operating system.

The problem solved by this project is to design and implement a reservation system that (a) grants a request only when the system will remain in a safe state, (b) queues requests that cannot be granted safely in a fair FIFO order, and (c) re-examines the queue automatically whenever seats are released.

This matters because it keeps the hall usable for all groups, avoids the manual work of untangling blocked bookings, and gives the coordinator a clear view of whether the current allocation is safe.

1.3 Motivation

The Banker's Algorithm is usually explained with abstract tables of processes P0–P4 and resources A, B and C, which makes it difficult for students to relate it to a real situation. A seat-reservation system is familiar to every student and maps naturally onto the algorithm: seats are resources, groups are processes, bookings are requests and cancellations are releases. Building the system turns the textbook algorithm into something that can be run, observed and tested.

The topic was also selected because it exercises several SPOS ideas together – safe and unsafe states, the need matrix, safe sequences, blocking of requests and fairness through FIFO ordering – while remaining small enough to implement completely in a single Java program without external libraries.

1.4 Objectives

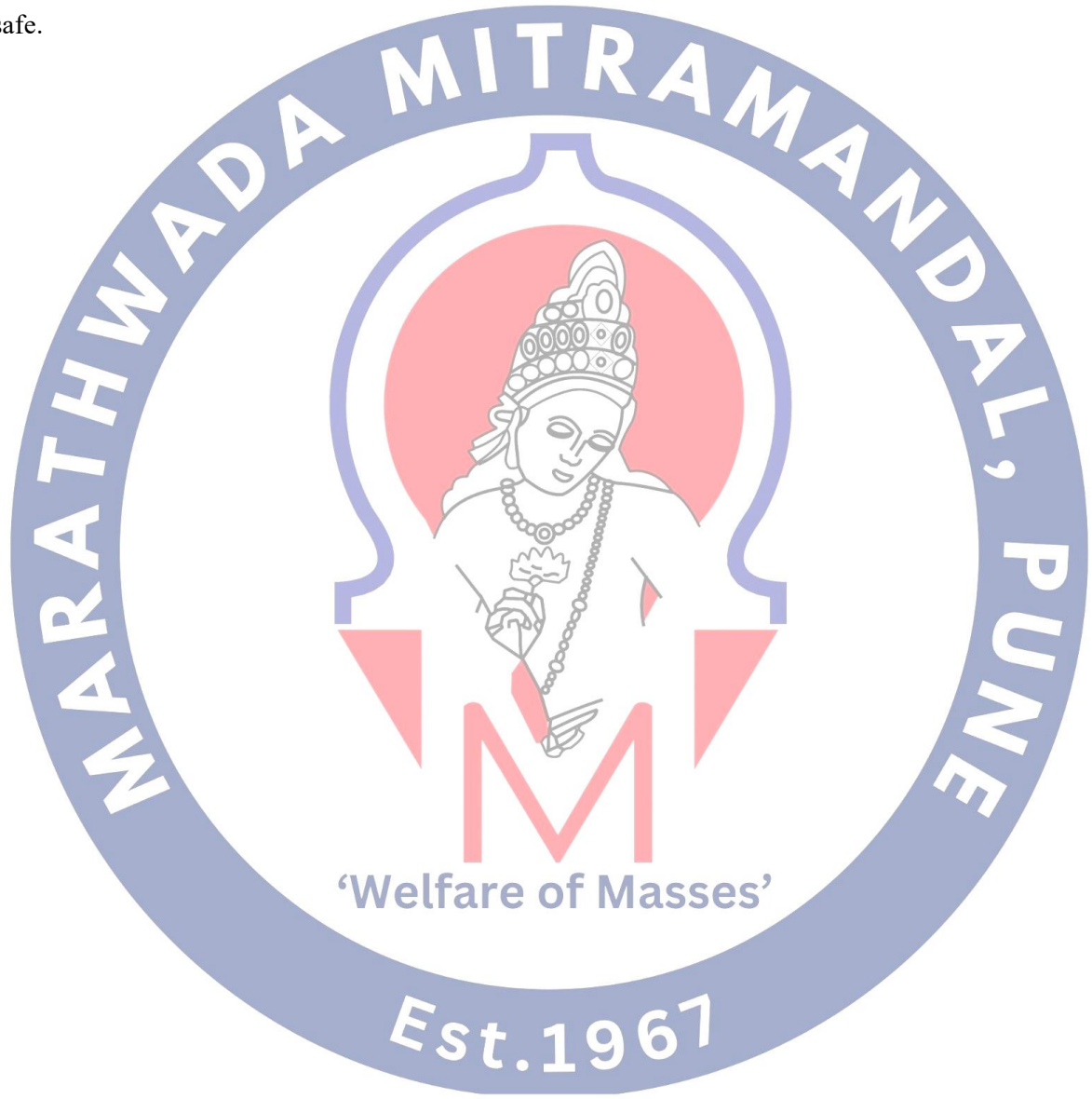
1. Model the seminar hall as a single-type resource with a user-defined capacity and model each group with a maximum claim, validating that $0 \leq$ maximum $\leq$ total seats.
2. Implement the Banker's safety algorithm so that it returns a safe sequence when the state is safe and reports failure otherwise.
3. Approve a booking only if the state that would result from it is safe, and verify this on normal and unsafe scenarios.
4. Maintain a FIFO waiting list for requests that exceed the free seats or lead to an unsafe state, and re-process it automatically after every cancellation.
5. Provide a cancellation operation that releases seats and correctly updates the available, allocation and need values.
6. Provide a dashboard that displays total, available, allocated, maximum and need values together with the SAFE/UNSAFE status and a safe sequence.
7. Validate every input (group number, seats to book, seats to cancel, menu choice, capacity and maximum claims).
8. Test the system with at least 15 test cases covering normal, boundary and invalid inputs and record the actual results.

1.5 Scope of the Project

Included: one seminar hall with a single type of resource (identical seats); any number of student groups chosen at start-up; booking, cancellation, dashboard and waiting-list operations; Banker's

safety checking; FIFO handling of waiting requests; console-based input and output; in-memory data for one session.

Excluded: multiple halls, dates or time slots; allocation of individual named seats; graphical user interface; database or file persistence; user authentication; multi-user or multi-threaded access; priorities between groups. The intended use is as an academic demonstration of deadlock avoidance and as a prototype for a hall coordinator who wants to check whether a proposed set of bookings is safe.



CHAPTER 2: THEORETICAL BACKGROUND

2.1 Selected SPOS Concept

The selected concept is **deadlock handling in operating systems, specifically deadlock avoidance with the Banker's Algorithm**. A deadlock is a situation in which a set of processes is blocked forever because each process holds some resources and waits for resources held by another process in the set. A deadlock can occur only if four conditions (the Coffman conditions) hold together:

- **Mutual exclusion** – a resource instance can be used by only one process at a time (a seat belongs to one group).
- **Hold and wait** – a process holds some resources while waiting for more (a group keeps its partial booking while waiting for extra seats).
- **No pre-emption** – resources cannot be taken away forcibly (seats are released only by the group itself).
- **Circular wait** – each process in a set waits for a resource held by the next one (every group waits for seats held by the others).

An operating system may prevent deadlock by breaking one of these conditions, avoid it by examining every request dynamically, or detect and recover from it after it happens. **Deadlock avoidance** needs advance information: each process declares its maximum demand. The system then grants a request only if the resulting state is **safe**. A state is safe if there is an order of the processes – a **safe sequence** – such that each process can obtain its remaining need from the free resources plus those released by the processes before it. An unsafe state is not necessarily a deadlock, but it gives no guarantee that deadlock can be avoided, so the Banker's Algorithm never allows the system to enter it. The algorithm was proposed by E. W. Dijkstra and generalised by A. N. Habermann; its name comes from a banker who never lends cash in a way that prevents satisfying all customers.

In SmartSem the concept is mapped onto the seminar hall as follows:

Table 2.1: Mapping of Banker's Algorithm concepts to the seminar hall

Operating-system concept	Seminar hall equivalent
Resource type with identical instances	Seats in the seminar hall (single type, total capacity entered by the user)
Process	Student group or department (e.g. Computer Dept, IT Dept, ENTC Dept)
Maximum claim of a process	Maximum seats the group may need for its event
Resource request	Booking request for some seats
Resource release	Cancellation of booked seats
Blocked process / request queue	FIFO waiting list of pending bookings
Safe state, safe sequence	An order in which every group can obtain its full quota and finish

2.2 Relevant Terminology

Table 2.2: Terms, meaning and the corresponding variables in the program

Term	Meaning in SmartSem	Variable / structure
Total seats	Capacity of the seminar hall	totalSeats
Available	Seats that are currently free; Available = Total − Σ Allocation	available
Max	Maximum seats a group may ever need	max[i]
Allocation	Seats currently booked by a group	allocation[i]
Need	Seats a group may still ask for; Need[i] = Max[i] − Allocation[i]	need[i]
Work	Temporary count of free seats used while testing safety	work (in getSafeSequence)
Finish	Flag showing that a group can complete and release its seats in the test	finish[i]
Safe sequence	Order of groups in which all can finish	sequence[ ] / int[] returned
Safe / Unsafe state	State with / without a safe sequence	getSafeSequence() != null
Request	A pair (group, seats) asking for seats	class Request
Pending seats	Seats of a group already waiting in the queue; prevents over-requesting	pendingSeats[g]
Waiting list	FIFO queue of requests that could not be granted	Queue<Request> waitingList
FIFO	First request to enter the queue is examined first	LinkedList / poll() / offer()

2.3 Algorithm(s) / Technique(s)

Assumptions. (1) There is a single resource type whose instances are identical. (2) Every group declares its maximum claim before it starts and never requests more than its remaining need. (3) A group that has obtained its maximum will use the seats and then release them in finite time. (4) Seats are not taken away from a group by force.

A. Safety Algorithm

1. Let Work = Available and Finish[i] = false for every group i.
2. Find a group i such that Finish[i] = false and Need[i] ≤ Work. If no such group exists, go to step 4.
3. Set Work = Work + Allocation[i], Finish[i] = true, append i to the safe sequence and go back to step 2.
4. If Finish[i] = true for all groups, the state is SAFE and the recorded order is a safe sequence; otherwise the state is UNSAFE.

B. Resource-Request (Booking) Algorithm

1. Let a group g request s seats. If s > Need[g] − Pending[g] or s ≤ 0, reject the request as invalid.
2. If the waiting list is not empty, add the request to the end of the waiting list (FIFO fairness).
3. If s > Available, add the request to the waiting list because the seats are not free yet.
4. Otherwise pretend to allocate: Available −= s, Allocation[g] += s, Need[g] −= s.
5. Run the safety algorithm on the pretended state, then restore the old values.
6. If the state is safe, perform the allocation for real (request approved); if it is unsafe, add the request to the waiting list.

C. Waiting-List Processing after a Cancellation

When seats are released, the queue length q is noted and exactly q requests are taken from the front one by one. For each request the seats are tested again with steps 3–6 above. A request that can now be granted safely is approved; a request that still cannot be granted is put back at the end of the queue, so the relative order of the remaining requests is preserved.

D. Worked Example

Hall capacity 10; Computer Dept (max 7), IT Dept (max 5), ENTC Dept (max 6). After Computer Dept books 2 seats and IT Dept books 3 seats, Available = 5, Allocation = (2, 3, 0) and Need = (5, 2, 6). The safety check gives Work = 5 → Computer (5 ≤ 5) → Work = 7 → IT (2 ≤ 7) → Work = 10 → ENTC (6 ≤ 10): the state is safe with sequence Computer, IT, ENTC. If ENTC now asks for 4 seats, the pretended state has Available = 1 and Need = (5, 2, 2); no group has Need ≤ 1, so the state is unsafe and the request is queued even though 5 seats are physically free.

E. Complexity

One safety check scans the n groups repeatedly until no further group can finish, which takes $O(n^2)$ time in the worst case and O(n) extra space. A booking needs one safety check; processing a waiting list of q requests needs at most q safety checks, i.e. $O(q \cdot n^2)$. For the small values of n used by a college this is instantaneous.

2.4 Technologies and Tools Used

Table 2.3: Technologies and tools

Item	Details	Purpose
Programming language	Java (JDK 8 or later; tested with OpenJDK 21)	Implementation of the whole application

Item	Details	Purpose
Compiler / launcher	javac and java (or the single-file launcher: java SeminarHallReservation.java)	Compile and run the program
Libraries	java.util only: Scanner, Queue, LinkedList	Console input, FIFO waiting list
Operating system	Windows 10/11, Linux (Ubuntu) or macOS	Platform to run the JVM
Terminal / IDE	Command prompt, shell, VS Code, IntelliJ IDEA, Eclipse or NetBeans	Editing, compiling, executing, taking screenshots
Documentation tools	Microsoft Word; diagrams and console screenshots as images	Preparation of this report



CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

3.1 Functional Requirements

Table 3.1: Functional requirements

ID	Requirement
FR1	Accept the total number of seminar hall seats and the number of student groups; reject non-positive values.
FR2	Accept a name and a maximum seat requirement for every group; reject a maximum that is negative or larger than the hall capacity.
FR3	Display a main menu with options Book, Cancel, Dashboard, Waiting List and Exit, and repeat it until Exit is chosen.
FR4	Validate the group number and the number of seats for every booking; a request must not exceed the group's remaining need after its already waiting seats.
FR5	Run the Banker's safety algorithm on every booking and approve it only if the resulting state is safe.
FR6	Place a request in the FIFO waiting list if it exceeds the available seats, would make the state unsafe, or if other requests are already waiting.
FR7	Allow a group to cancel seats up to the number it currently holds and update available, allocation and need.
FR8	After every cancellation, re-examine all waiting requests in FIFO order and approve those that are now safe.
FR9	Display a dashboard with total, available, allocated, maximum and need values, the SAFE/UNSAFE status, a safe sequence and the number of waiting requests.
FR10	Display the waiting list showing the group and the seats requested by each pending request.
FR11	Report invalid menu choices and invalid numbers with clear messages and continue running.

Non-functional requirements: the program should be easy to use through a simple numbered menu (usability), must keep the system in a safe state at all times (correctness), should respond instantly for the group counts used in a college (performance) and should run on any platform with a Java runtime (portability).

3.2 Hardware and Software Requirements

Hardware: any personal computer with a dual-core processor of 1 GHz or faster, at least 2 GB RAM (4 GB recommended) and about 500 MB of free disk space (mostly for the JDK). A keyboard and a display for the console are required.

Software: Windows 10/11, Ubuntu Linux or macOS; Java Development Kit (JDK) 8 or later (the project was executed with OpenJDK 21); a terminal or command prompt; optionally an IDE such as VS Code, IntelliJ IDEA, Eclipse or NetBeans. No external libraries are needed.

3.3 System Architecture / Block Diagram/FlowChart

Figure 3.1 shows the architecture of SmartSem. The user interacts only with the console menu. The

menu passes each choice to one of three groups of functions: the booking function, the cancellation function, and the display functions (dashboard and waiting list). Booking and the waiting-list processor both depend on the Banker's Algorithm engine, which consists of the functions getSafeSequence() and isSafeRequest(). All functions read and update a common set of static data: the arrays max[ ], allocation[ ], need[ ] and pendingSeats[ ], the counters totalSeats and available, and the FIFO queue waitingList.

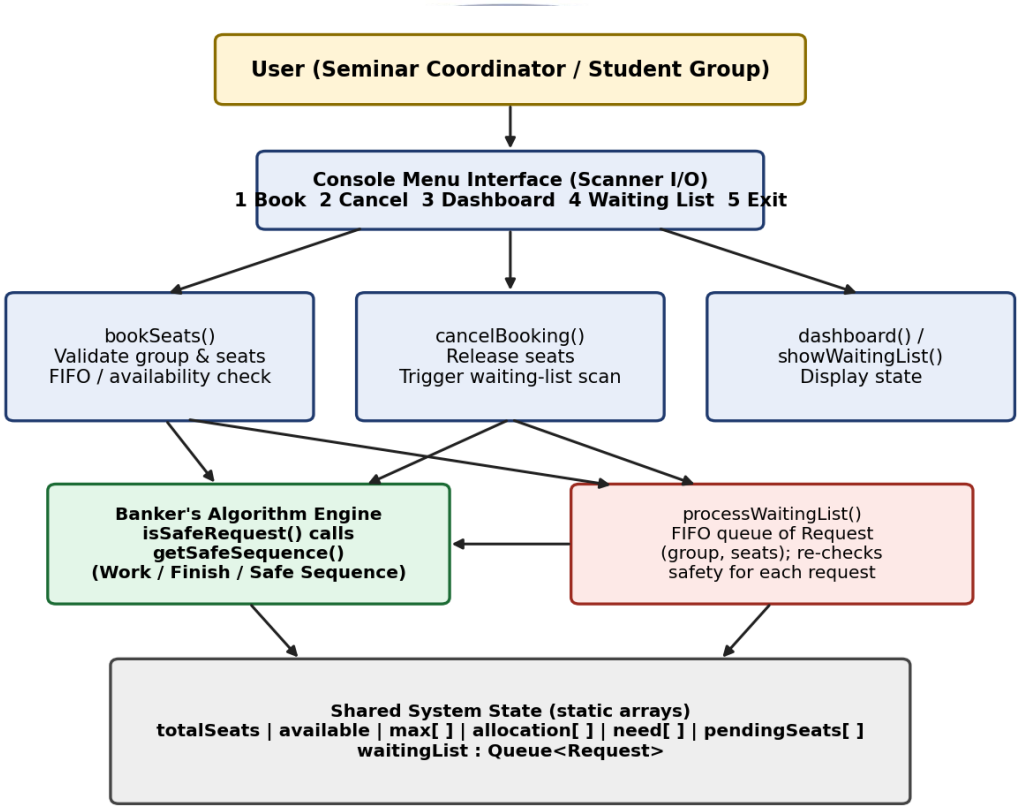


Fig. 3.1: System architecture / block diagram of SmartSem

The flow of information for a booking request is shown in Figure 3.2. The request is first validated; if other requests are already waiting, or if the seats are not free, it is queued. Otherwise the seats are allocated temporarily, the safety algorithm is run, the temporary allocation is rolled back, and the request is either committed (safe) or queued (unsafe). When a cancellation releases seats,

processWaitingList() feeds each queued request through the same safety test.

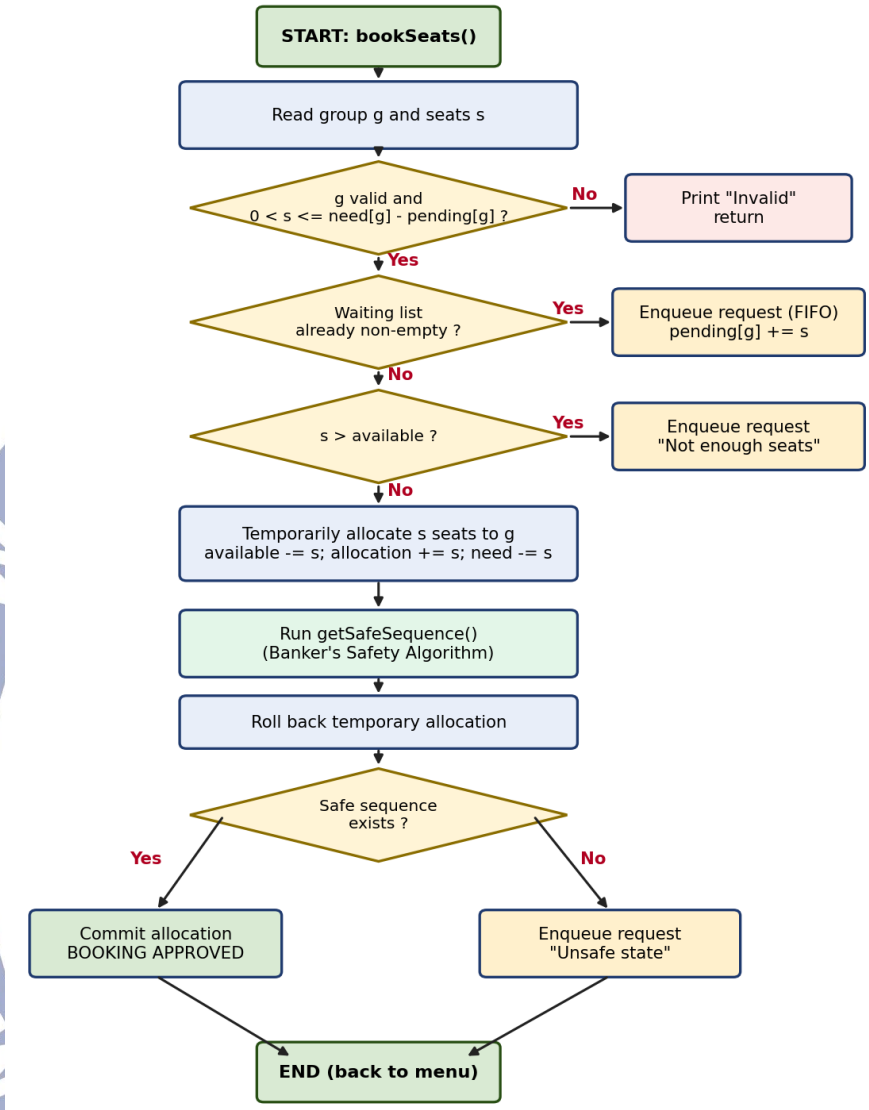


Fig. 3.2: Flowchart of the seat booking request

3.5 Algorithm / Pseudocode

Algorithm 1: Safety check – getSafeSequence()

```
1.  work   <- available
2.  finish[0..n-1] <- false ; count <- 0
3.  while count < n:
4.      found <- false
5.      for i <- 0 to n-1:
6.          if finish[i] = false AND need[i] <= work:
7.              work <- work + allocation[i]
8.              finish[i] <- true
9.              sequence[count] <- i ; count <- count + 1
10.             found <- true
11.     if found = false: return NULL          // unsafe
12. return sequence                           // safe
```

Algorithm 2: Test a request – isSafeRequest(g, seats)

```
1.  available <- available - seats ; allocation[g] <- allocation[g] + seats
2.  need[g]   <- need[g] - seats
3.  safe <- (getSafeSequence() != NULL)
4.  restore available, allocation[g], need[g]   // undo the trial
5.  return safe
```

Algorithm 3: Booking – bookSeats()

```
1.  read g (1..n) and seats
2.  if g is invalid:                                print "Invalid group number"; return
3.  if seats <= 0 OR seats > need[g] - pending[g]: print "Invalid request";    return
4.  if waitingList is not empty:    enqueue(g, seats); pending[g] += seats; return
5.  if seats > available:           enqueue(g, seats); pending[g] += seats; return
6.  if isSafeRequest(g, seats):
7.      available -= seats ; allocation[g] += seats ; need[g] -= seats
8.      print "BOOKING APPROVED"
9.  else:
10.     enqueue(g, seats) ; pending[g] += seats
11.     print "Request would cause an unsafe state"
12. return
```

Algorithm 4: Cancellation and waiting list – cancelBooking(), processWaitingList()

```
1.  read g ; if g invalid or allocation[g] = 0: print message; return
2.  read seats ; if seats <= 0 OR seats > allocation[g]: print "Invalid cancellation";
return
3.  allocation[g] -= seats ; need[g] += seats ; available += seats
4.  size <- length(waitingList)
5.  repeat size times:
6.      r <- dequeue(waitingList) ; pending[r.group] -= r.seats
7.      if r.seats <= available AND isSafeRequest(r.group, r.seats):
8.          available -= r.seats ; allocation[r.group] += r.seats
9.          need[r.group] -= r.seats ; print "WAITING REQUEST APPROVED"
10.     else:
11.         pending[r.group] += r.seats ; enqueue(waitingList, r)
```



CHAPTER 4: IMPLEMENTATION

4.1 Implementation Approach

The solution was developed incrementally as a single Java class, SeminarHallReservation, using static fields and static methods so that all functions share one set of data. The Banker's Algorithm is represented directly by the data structures of the textbook: the arrays max[ ], allocation[ ] and need[ ] hold one value per group, and the integer available holds the free seats. Because the seats are of a single type, each of these is a simple integer rather than a vector, so the safety algorithm needs only a scalar work value.

The main design decisions were:

- **Trial allocation with rollback.** isSafeRequest() subtracts the seats, runs getSafeSequence(), and then restores the old values, so a request can be tested without permanently changing the state.
- **Need is derived.** need[i] is initialised to max[i] and then kept equal to max[i] − allocation[i] by every booking and cancellation.
- **pendingSeats[ ] array.** It stores the seats of a group that are already waiting in the queue, so a group cannot queue more seats than its remaining need.
- **FIFO fairness.** A new request joins the waiting list whenever the list is not empty, so it cannot overtake an older request.
- **Fixed-size scan.** processWaitingList() takes exactly as many requests from the queue as it held at the start, so unsatisfied requests that are re-added at the tail are not examined twice in the same scan.
- **Only java.util** (Scanner, Queue, LinkedList) is used, so the program runs on any standard JDK.

The development order was: (1) data model and start-up input; (2) getSafeSequence(); (3) isSafeRequest() and bookSeats(); (4) cancelBooking(); (5) waiting list with processWaitingList(); (6) dashboard and menu; (7) testing and correction of boundary conditions.

4.2 Module Description

Table 4.1: Modules (functions) and their responsibilities

Module / function	Responsibility
Static data (totalSeats, available, n, groups[ ], max[ ], allocation[ ], need[ ], pendingSeats[ ])	Holds the Banker's Algorithm state shared by all functions.

Module / function	Responsibility
class Request	Represents a pending booking: the group index and the number of seats requested.
getSafeSequence()	Implements the safety algorithm; returns the safe sequence as an int array or null if the state is unsafe.
isSafeRequest(g, seats)	Temporarily allocates seats to group g, checks safety, restores the state and returns true or false.
bookSeats()	Reads a booking, validates it, applies FIFO and availability rules and the Banker's check, then approves or queues the request.
cancelBooking()	Reads a cancellation, validates it, releases the seats and calls processWaitingList().
processWaitingList()	Re-examines every waiting request in FIFO order after seats are released and approves those that are safe.
dashboard()	Prints total and available seats, the allocation/maximum/need table, the SAFE/UNSAFE status, a safe sequence and the queue length.
showWaitingList()	Prints every pending request with its group and seat count.
main()	Reads the capacity, group names and maximum claims, validates them and runs the menu loop until Exit.

4.3 Important Source Code with Explanation

(a) Data structures and the Request class

```
    static int totalSeats, available, n;
    static String[] groups;
    static int[] max, allocation, need, pendingSeats;

    static Queue<Request> waitingList = new LinkedList<>();

    static class Request {
        int group, seats;

        Request(int group, int seats) {
            this.group = group;
            this.seats = seats;
        }
    }
```

The six arrays and counters form the Banker's state. A Request object stores the group index and the number of seats, and the LinkedList-based Queue gives FIFO behaviour through offer() (add at tail) and poll() (remove from head).

MARATHWADA MITRAMANDAL
'Welfare of Masses'
Est.1967

(b) Safety algorithm

```
// Banker's Algorithm: find a safe sequence
static int[] getSafeSequence() {
    int work = available;
    boolean[] finish = new boolean[n];
    int[] sequence = new int[n];
    int count = 0;

    while (count < n) {
        boolean found = false;

        for (int i = 0; i < n; i++) {
            if (!finish[i] && need[i] <= work) {
                work += allocation[i];
                finish[i] = true;
                sequence[count++] = i;
                found = true;
            }
        }

        if (!found) {
            return null;
        }
    }

    return sequence;
}
```

work starts as the number of free seats. Each pass of the for loop looks for an unfinished group whose need does not exceed work; such a group can finish, so its allocation is added back to work and its index is appended to the sequence. If a complete pass finds no group (found is false) the state is unsafe and null is returned; otherwise all n groups are placed in the sequence.

(c) Testing a request with a trial allocation

```
// Check a request by temporarily allocating seats
static boolean isSafeRequest(int g, int seats) {
    available -= seats;
    allocation[g] += seats;
    need[g] -= seats;

    boolean safe = getSafeSequence() != null;

    // Restore temporary allocation
    available += seats;
    allocation[g] -= seats;
    need[g] += seats;

    return safe;
}
```

The request is applied to available, allocation and need, the safety algorithm is run, and the three values are restored. The real state is therefore never changed by a test.

(d) Booking decision

```
        // Respect requests already waiting in the queue
        if (!waitingList.isEmpty()) {
            waitingList.offer(new Request(g, seats));
```

```
            pendingSeats[g] += seats;

            System.out.println("Added to waiting list (FIFO).");
            return;
        }

        if (seats > available) {
            waitingList.offer(new Request(g, seats));
            pendingSeats[g] += seats;

            System.out.println("Not enough seats. Added to waiting list.");
            return;
        }

        if (isSafeRequest(g, seats)) {
            available -= seats;
            allocation[g] += seats;
            need[g] -= seats;

            System.out.println("BOOKING APPROVED!");
            System.out.println("Banker's Algorithm: SAFE STATE.");
        } else {
            waitingList.offer(new Request(g, seats));
            pendingSeats[g] += seats;

            System.out.println("Request would cause an unsafe state.");
            System.out.println("Added to waiting list.");
        }
```

After the input has been validated, the request is queued if older requests are waiting (FIFO) or if there are not enough free seats. Only then is the Banker's check applied: a safe request is committed, an unsafe one is queued with a message.

(e) Releasing seats and processing the waiting list

```
    // Process the waiting list in FIFO order
    static void processWaitingList() {
        if (waitingList.isEmpty()) {
            return;
        }

        System.out.println("\nChecking waiting list...");

        int size = waitingList.size();

        for (int k = 0; k < size; k++) {
            Request r = waitingList.poll();
            int g = r.group;

            // Temporarily remove this request's pending claim
            pendingSeats[g] -= r.seats;

            if (r.seats <= available
                    && isSafeRequest(g, r.seats)) {

                available -= r.seats;
                allocation[g] += r.seats;
                need[g] -= r.seats;

                System.out.println("WAITING REQUEST APPROVED: "
                        + groups[g] + " - " + r.seats + " seats");
            } else {
                // Keep the request in the queue
                pendingSeats[g] += r.seats;
                waitingList.offer(r);
            }
```

```
        }

        if (waitingList.isEmpty()) {
            System.out.println("Waiting list is empty.");
        }
    }
```

The loop runs exactly size times. Each request is removed, its pending claim is withdrawn, and the request is approved if the seats are free and the state remains safe; otherwise its claim is restored and the request is re-added at the tail. cancelBooking() calls this method after it has increased available and need and decreased allocation.

(f) Complete source code – SeminarHallReservation.java

```
import java.util.*;

public class SeminarHallReservation {

    static Scanner sc = new Scanner(System.in);

    static int totalSeats, available, n;
    static String[] groups;
    static int[] max, allocation, need, pendingSeats;

    static Queue<Request> waitingList = new LinkedList<>();

    static class Request {
        int group, seats;

        Request(int group, int seats) {
            this.group = group;
            this.seats = seats;
        }
    }

    // Banker's Algorithm: find a safe sequence
    static int[] getSafeSequence() {
        int work = available;
        boolean[] finish = new boolean[n];
        int[] sequence = new int[n];
        int count = 0;

        while (count < n) {
            boolean found = false;

            for (int i = 0; i < n; i++) {
                if (!finish[i] && need[i] <= work) {
                    work += allocation[i];
                    finish[i] = true;
                    sequence[count++] = i;
                    found = true;
                }
            }

            if (!found) {
                return null;
            }
        }

        return sequence;
    }

    // Check a request by temporarily allocating seats
    static boolean isSafeRequest(int g, int seats) {
        available -= seats;
        allocation[g] += seats;
        need[g] -= seats;
```

```
        boolean safe = getSafeSequence() != null;

        // Restore temporary allocation
        available += seats;
        allocation[g] -= seats;
        need[g] += seats;

        return safe;
    }

    // Display the safe/unsafe dashboard
    static void dashboard() {
        System.out.println("\n========== SMARTSEM DASHBOARD ==========");
        System.out.println("Total hall seats : " + totalSeats);
        System.out.println("Available seats  : " + available);

        System.out.println("\nGroup\tAllocated\tMaximum\tNeed");

        for (int i = 0; i < n; i++) {
            System.out.println(groups[i] + "\t"
                    + allocation[i] + "\t\t"
                    + max[i] + "\t" + need[i]);
        }

        int[] seq = getSafeSequence();

        if (seq != null) {
            System.out.print("\nSystem Status: SAFE\nSafe Sequence: ");

            for (int i : seq) {
                System.out.print(groups[i] + " ");
            }

            System.out.println();
        } else {
            System.out.println("\nSystem Status: UNSAFE");
        }

        System.out.println("Waiting requests: " + waitingList.size());
        System.out.println("========================================");
    }

    // Process the waiting list in FIFO order
    static void processWaitingList() {
        if (waitingList.isEmpty()) {
            return;
        }

        System.out.println("\nChecking waiting list...");

        int size = waitingList.size();

        for (int k = 0; k < size; k++) {
            Request r = waitingList.poll();
            int g = r.group;

            // Temporarily remove this request's pending claim
            pendingSeats[g] -= r.seats;

            if (r.seats <= available
                    && isSafeRequest(g, r.seats)) {

                available -= r.seats;
                allocation[g] += r.seats;
                need[g] -= r.seats;

                System.out.println("WAITING REQUEST APPROVED: "
                        + groups[g] + " - " + r.seats + " seats");
            } else {
                // Keep the request in the queue
                pendingSeats[g] += r.seats;
                waitingList.offer(r);
            }
        }
```

```
        if (waitingList.isEmpty()) {
            System.out.println("Waiting list is empty.");
        }
    }

    // Request a new reservation
    static void bookSeats() {
        System.out.print("Enter group number (1-" + n + "): ");
        int g = sc.nextInt() - 1;

        if (g < 0 || g >= n) {
            System.out.println("Invalid group number!");
            return;
        }

        System.out.print("Enter seats to book: ");
        int seats = sc.nextInt();

        if (seats <= 0 || seats > need[g] - pendingSeats[g]) {
            System.out.println("Invalid request! Check remaining need.");
            return;
        }

        // Respect requests already waiting in the queue
        if (!waitingList.isEmpty()) {
            waitingList.offer(new Request(g, seats));
            pendingSeats[g] += seats;

            System.out.println("Added to waiting list (FIFO).");
            return;
        }

        if (seats > available) {
            waitingList.offer(new Request(g, seats));
            pendingSeats[g] += seats;

            System.out.println("Not enough seats. Added to waiting list.");
            return;
        }

        if (isSafeRequest(g, seats)) {
            available -= seats;
            allocation[g] += seats;
            need[g] -= seats;

            System.out.println("BOOKING APPROVED!");
            System.out.println("Banker's Algorithm: SAFE STATE.");
        } else {
            waitingList.offer(new Request(g, seats));
            pendingSeats[g] += seats;

            System.out.println("Request would cause an unsafe state.");
            System.out.println("Added to waiting list.");
        }
    }

    // Cancel a booking and release seats
    static void cancelBooking() {
        System.out.print("Enter group number (1-" + n + "): ");
        int g = sc.nextInt() - 1;

        if (g < 0 || g >= n) {
            System.out.println("Invalid group number!");
            return;
        }

        if (allocation[g] == 0) {
            System.out.println("This group has no seats to release.");
            return;
        }

        System.out.println("Currently booked seats: " + allocation[g]);
        System.out.print("Enter seats to cancel: ");
        int seats = sc.nextInt();
```

```
        if (seats <= 0 || seats > allocation[g]) {
            System.out.println("Invalid cancellation!");
            return;
        }

        allocation[g] -= seats;
        need[g] += seats;
        available += seats;

        System.out.println("CANCELLATION SUCCESSFUL!");
        System.out.println(seats + " seats released.");

        processWaitingList();
    }

    // Display waiting requests
    static void showWaitingList() {
        System.out.println("\n========== WAITING LIST ==========");

        if (waitingList.isEmpty()) {
            System.out.println("No pending requests.");
        } else {
            for (Request r : waitingList) {
                System.out.println(groups[r.group]
                        + " requested " + r.seats + " seats");
            }
        }

        System.out.println("==================================");
    }

    public static void main(String[] args) {

        System.out.println("==================================");
        System.out.println(" SMARTSEM - COLLEGE HALL SYSTEM");
        System.out.println(" Banker's Algorithm Based");
        System.out.println("==================================");

        System.out.print("Enter total seminar hall seats: ");
        totalSeats = sc.nextInt();

        System.out.print("Enter number of student groups: ");
        n = sc.nextInt();

        if (totalSeats <= 0 || n <= 0) {
            System.out.println("Invalid input!");
            sc.close();
            return;
        }

        groups = new String[n];
        max = new int[n];
        allocation = new int[n];
        need = new int[n];
        pendingSeats = new int[n];

        for (int i = 0; i < n; i++) {
            sc.nextLine();

            System.out.print("Enter group/department name " + (i + 1) + ": ");
            groups[i] = sc.nextLine();

            System.out.print("Maximum seats needed by "
                    + groups[i] + ": ");
            max[i] = sc.nextInt();

            if (max[i] < 0 || max[i] > totalSeats) {
                System.out.println("Invalid maximum seat requirement!");
                sc.close();
                return;
            }

            // Initially, no seats are booked
            allocation[i] = 0;
            need[i] = max[i];
```

```
        }

        available = totalSeats;

        int choice;

        do {
            System.out.println("\n========== MAIN MENU ==========");
            System.out.println("1. Book Seminar Hall Seats");
            System.out.println("2. Cancel Booking");
            System.out.println("3. View Safe/Unsafe Dashboard");
            System.out.println("4. View Waiting List");
            System.out.println("5. Exit");
            System.out.print("Enter your choice: ");

            choice = sc.nextInt();

            switch (choice) {
                case 1:
                    bookSeats();
                    break;

                case 2:
                    cancelBooking();
                    break;

                case 3:
                    dashboard();
                    break;

                case 4:
                    showWaitingList();
                    break;

                case 5:
                    System.out.println("Thank you for using SmartSem!");
                    break;

                default:
                    System.out.println("Invalid choice!");
            }

        } while (choice != 5);

        sc.close();
    }
}
```

4.4 Execution Procedure

1. Install a JDK (version 8 or later) and make sure the commands java and javac are available in the terminal.
2. Save the program as SeminarHallReservation.java; the file name must match the public class name.
3. Open a terminal in the folder that contains the file.
4. Compile the program: javac SeminarHallReservation.java
5. Run it: java SeminarHallReservation (with JDK 11 or later the two steps can be combined as: java SeminarHallReservation.java).
6. Enter the hall capacity, the number of groups, and for each group its name and maximum seats (for example 10, 3, Computer Dept 7, IT Dept 5, ENTC Dept 6).

7. Use the menu: 1 – Book, 2 – Cancel, 3 – Dashboard, 4 – Waiting List, 5 – Exit.

```
javac SeminarHallReservation.java
java SeminarHallReservation
```

4.5 Input and Output Screenshots

All screenshots below were produced by running the program with a hall capacity of 10 seats and three groups: Computer Dept (maximum 7), IT Dept (maximum 5) and ENTC Dept (maximum 6).

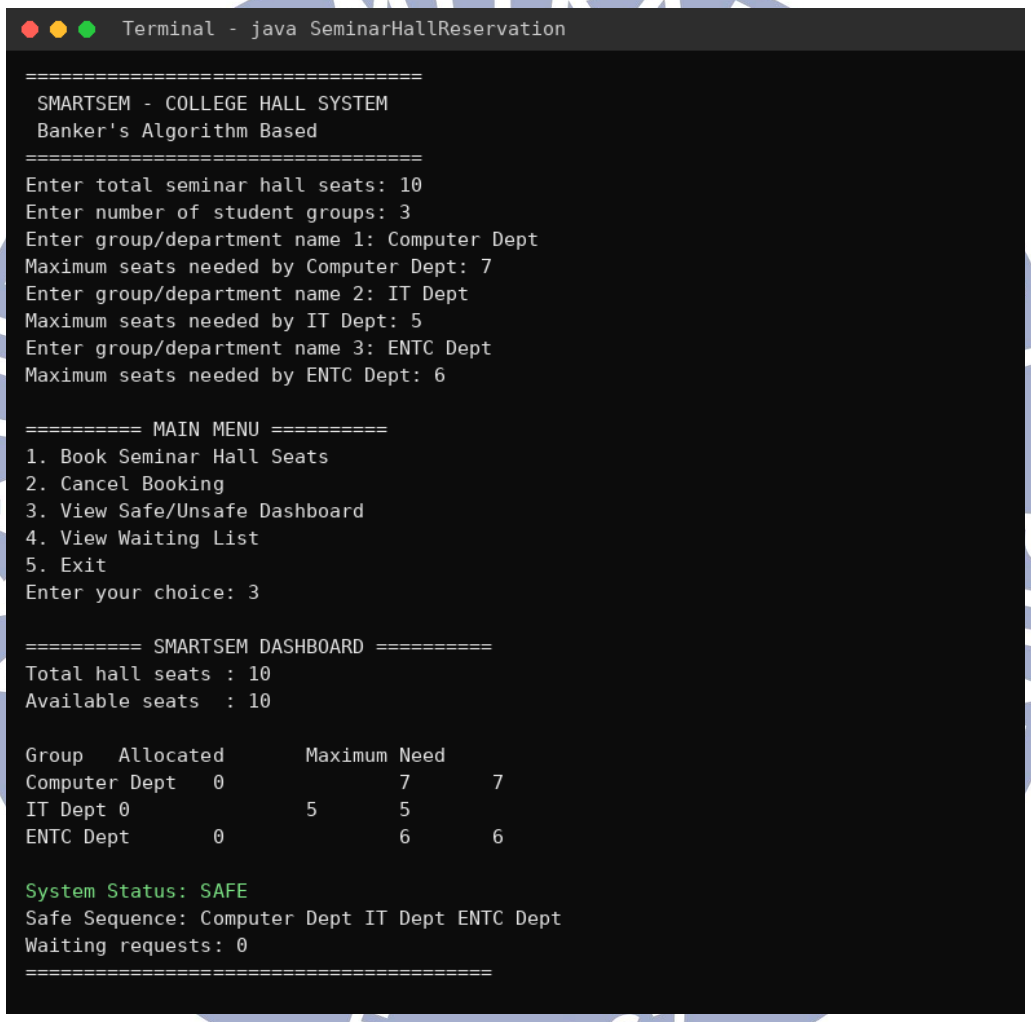


Fig. 4.1: Start-up inputs, main menu and the initial dashboard (all seats free, system SAFE)

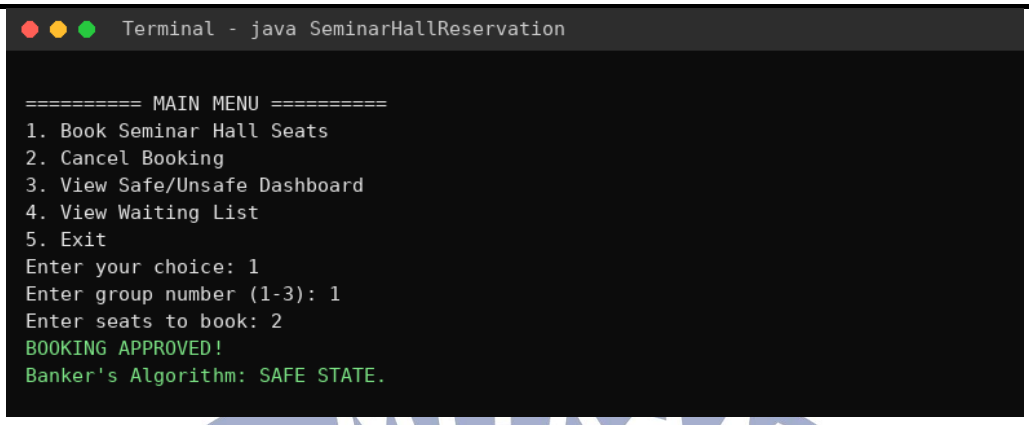


Fig. 4.2: Safe bookings – Computer Dept books 2 seats and IT Dept books 3 seats; both approved

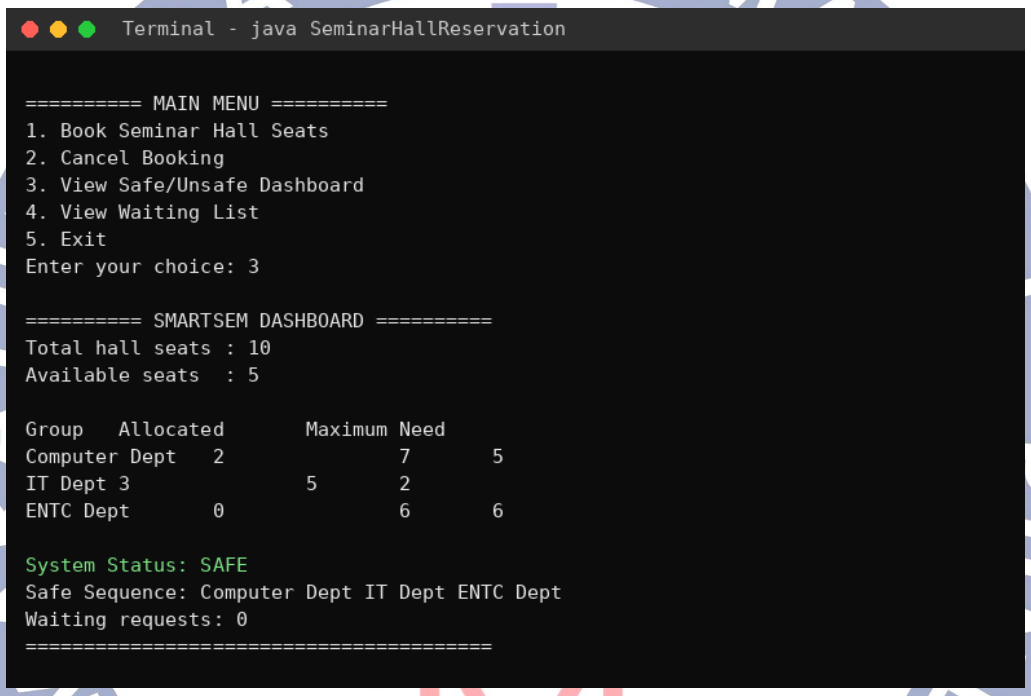


Fig. 4.3: Dashboard after the two bookings (Available = 5, SAFE, safe sequence shown)

'Welfare of Masses'
Est.1967

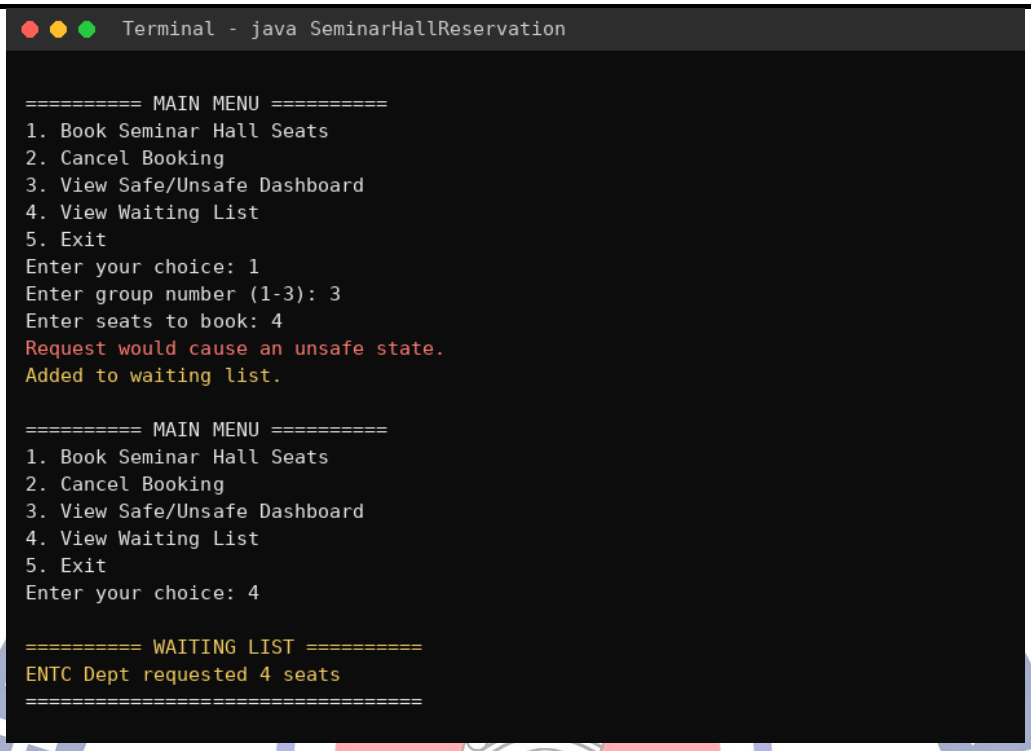


Fig. 4.4: ENTC Dept requests 4 seats; the request would be unsafe, so it is added to the waiting list

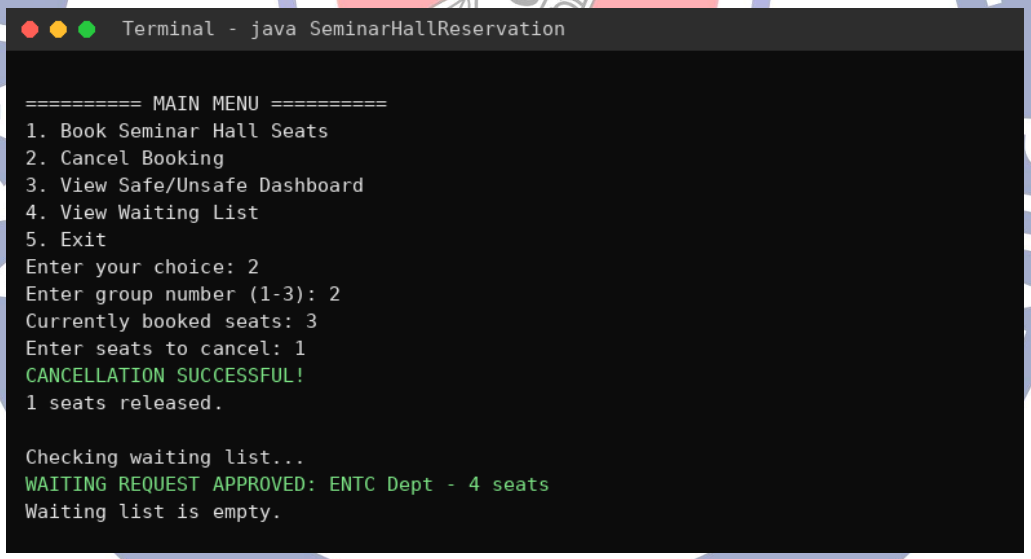


Fig. 4.5: IT Dept cancels 1 seat; the waiting request of ENTC Dept is approved automatically

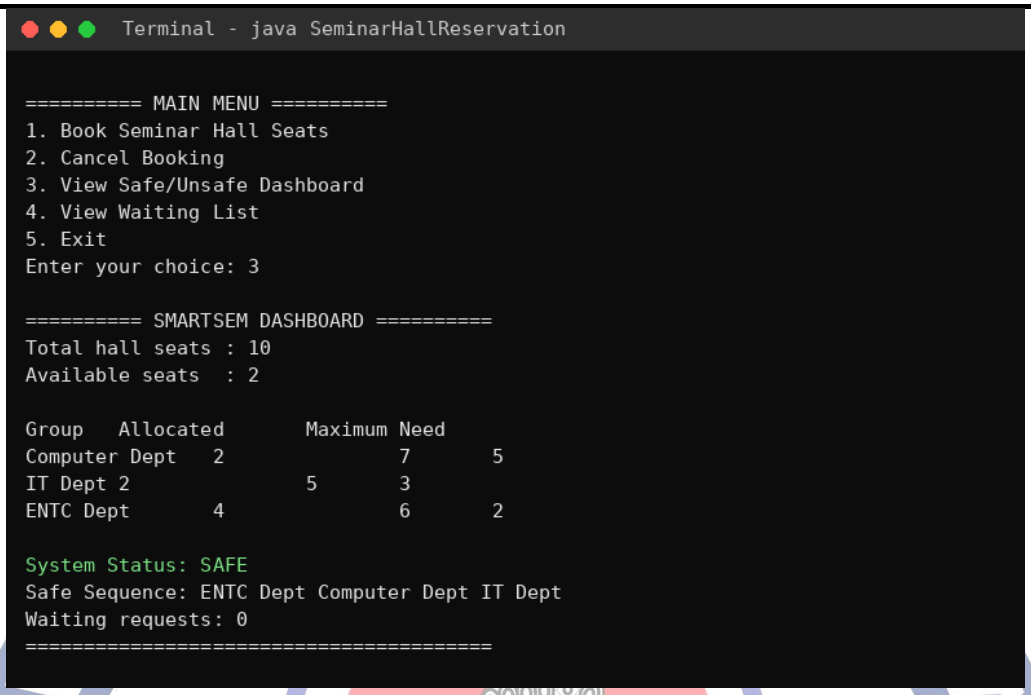


Fig. 4.6: Dashboard after the waiting request is approved (new safe sequence)

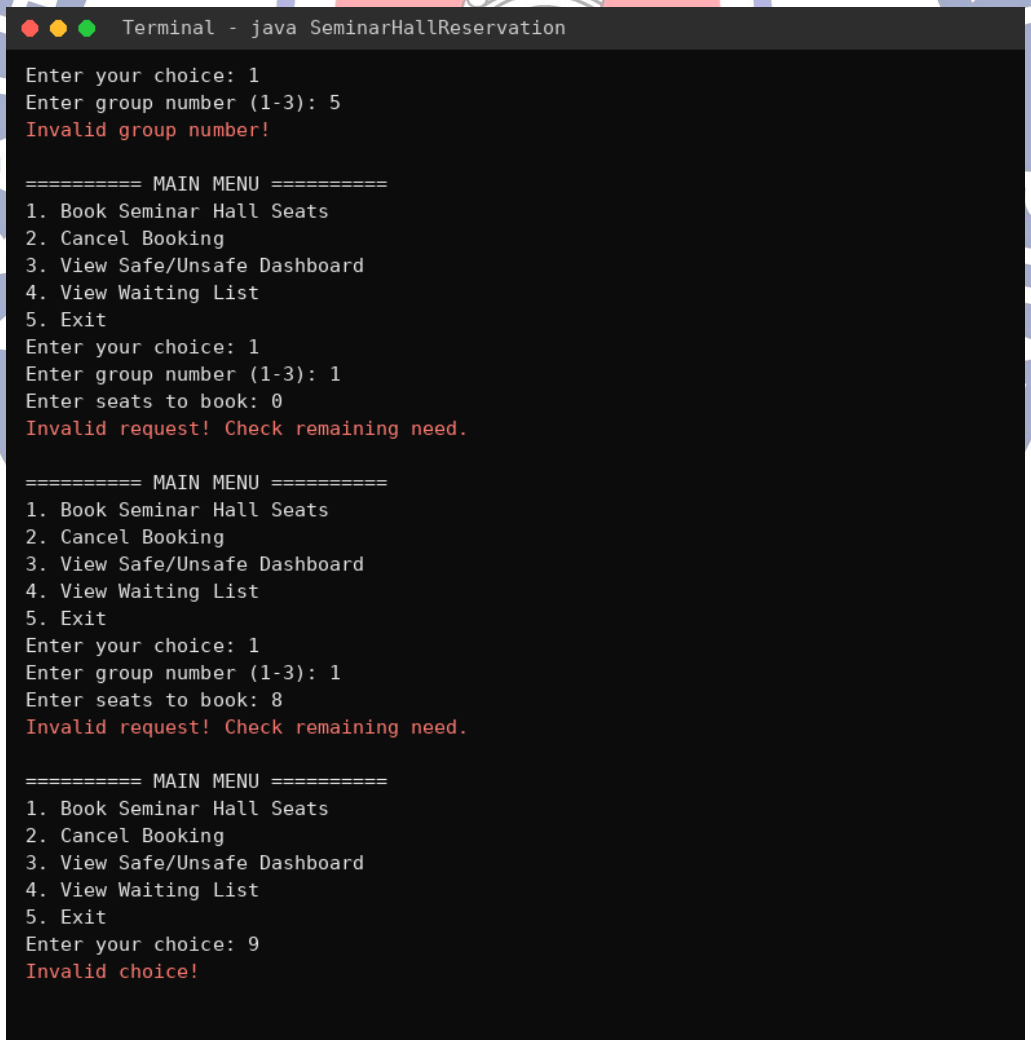


Fig. 4.7: Invalid inputs – wrong group number, zero seats, seats above need, invalid menu choice

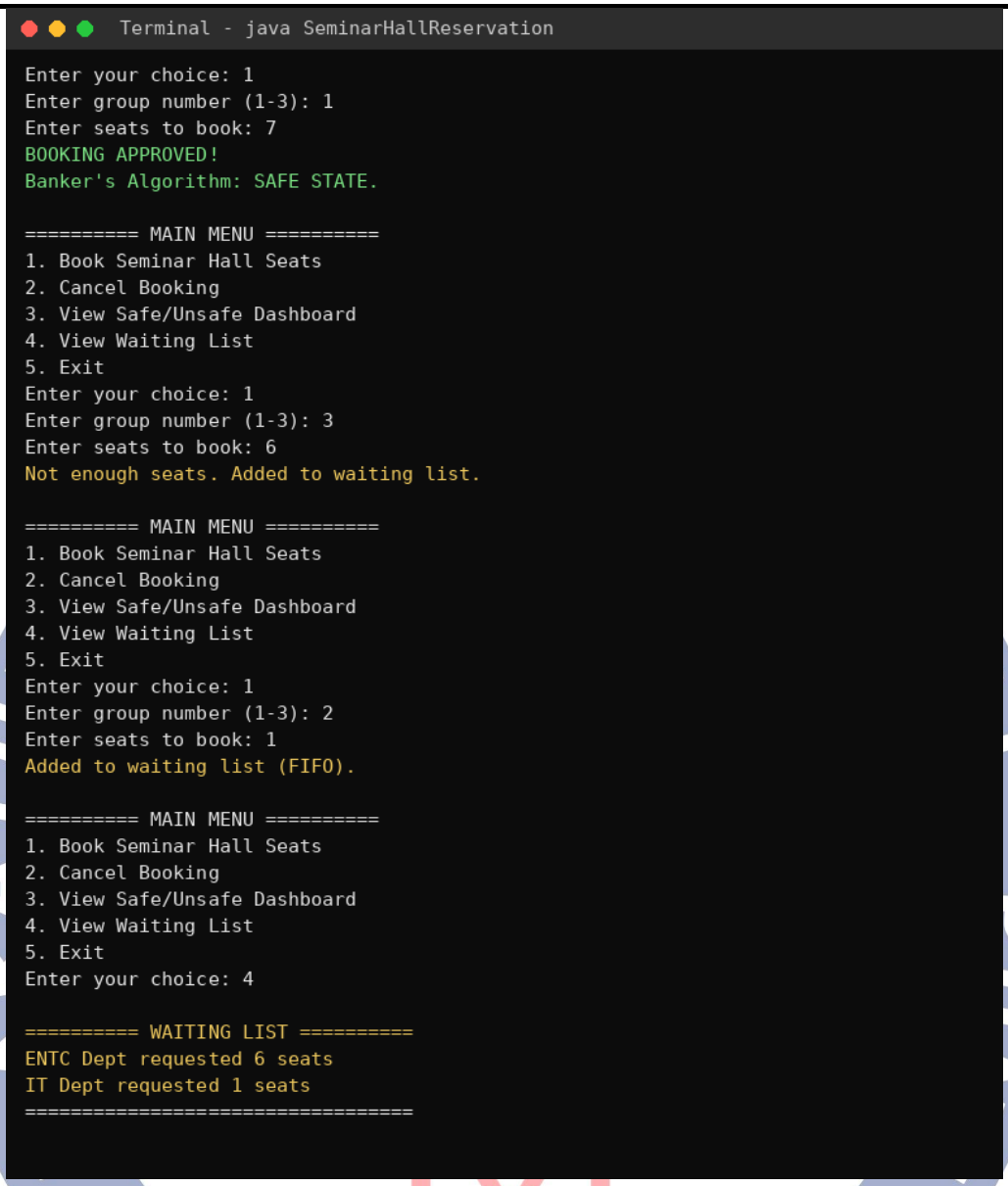


Fig. 4.8: Computer Dept books 7 seats; ENTC's request for 6 seats exceeds the 3 free seats and IT's request is queued behind it (FIFO); waiting list displayed

'Welfare of Masses'
Est.1967

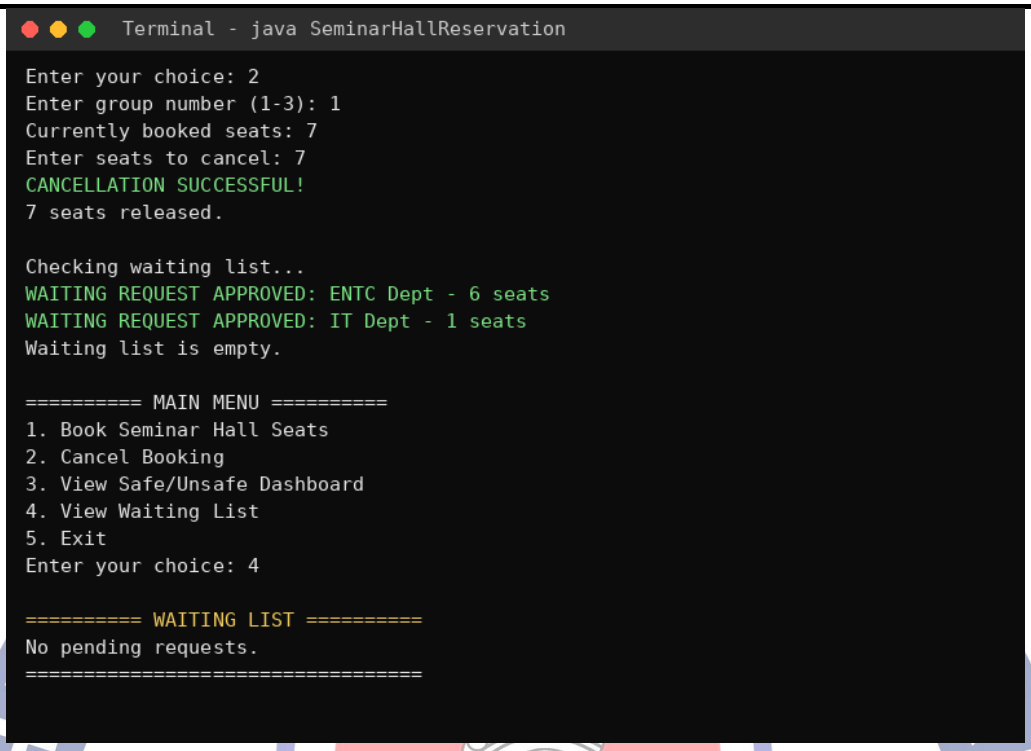


Fig. 4.9: Cancellation of 7 seats releases the hall; both waiting requests are approved in FIFO order



Fig. 4.10: Invalid start-up input – zero capacity, and a maximum claim larger than the hall capacity

'Welfare of Masses'
Est.1967

CHAPTER 5: TESTING AND RESULTS

5.1 Test Cases

Unless stated otherwise the hall capacity is 10 and the groups are Computer Dept (max 7), IT Dept (max 5) and ENTC Dept (max 6). The expected output was written before execution and the actual output was taken from running the program.

Table 5.1: Test cases

Test Case ID	Input / Scenario	Expected Output	Actual Output	Status
TC01	Start-up: capacity 10, 3 groups; open dashboard	Menu appears; Available 10; allocation 0; SAFE; sequence Computer, IT, ENTC	Same as expected	Pass
TC02	Computer Dept books 2 seats	BOOKING APPROVED; SAFE STATE	BOOKING APPROVED; Banker's Algorithm: SAFE STATE	Pass
TC03	IT Dept books 3 seats; open dashboard	Approved; Available 5; allocation 2/3/0; need 5/2/6; SAFE	Same as expected	Pass
TC04	Group 1 books 8 seats (max 7)	Invalid request message	Invalid request! Check remaining need.	Pass
TC05	Group 1 books 0 seats	Invalid request message	Invalid request! Check remaining need.	Pass
TC06	Booking with group number 5 (only 3 groups)	Invalid group number message	Invalid group number!	Pass
TC07	After TC03, ENTC Dept books 4 seats (unsafe case)	Request refused and queued as unsafe	Request would cause an unsafe state. Added to waiting list.	Pass
TC08	View waiting list after TC07	Shows ENTC Dept – 4 seats	ENTC Dept requested 4 seats	Pass
TC09	IT Dept cancels 1 seat (after TC07)	Cancellation succeeds; waiting request approved; list empty; SAFE with sequence ENTC, Computer, IT	CANCELLATION SUCCESSFUL; WAITING REQUEST APPROVED; ENTC Dept – 4 seats; Waiting list is empty; Available 2; SAFE: ENTC, Computer, IT	Pass
TC10	Boundary: ENTC Dept books 6 seats (its full maximum) on an empty hall	Approved	BOOKING APPROVED; SAFE STATE	Pass
TC11	ENTC Dept (holds 6) cancels 7 seats	Invalid cancellation	Invalid cancellation!	Pass
TC12	Cancel for a group that holds no seats	Message that nothing can be released	This group has no seats to release.	Pass
TC13	Computer Dept books 7 (approved), then ENTC Dept books 6 with only 3 free	ENTC request queued: not enough seats	Not enough seats. Added to waiting list.	Pass
TC14	With ENTC request waiting, IT Dept books 1 seat (1 ≤ 3 free)	Queued behind the older request (FIFO)	Added to waiting list (FIFO).	Pass
TC15	Computer Dept cancels all 7 seats (queue: ENTC 6, IT 1)	Both requests approved in FIFO order; list empty	WAITING REQUEST APPROVED: ENTC Dept – 6 seats; IT Dept – 1 seats; Waiting list is empty; Available 3; SAFE: ENTC, Computer, IT	Pass
TC16	Menu choice 9	Invalid choice message; menu repeats	Invalid choice!	Pass
TC17	Capacity 0, groups 3	Invalid input; program ends	Invalid input!	Pass
TC18	Group maximum 12 with capacity 10	Invalid maximum seat requirement; program ends	Invalid maximum seat requirement!	Pass

Test Case ID	Input / Scenario	Expected Output	Actual Output	Status
TC19	Non-numeric text (abc) typed at the menu prompt	Error message and menu continues	Program stops with java.util.InputMismatchException	Fail

5.2 Actual Results

Normal inputs. Safe bookings (TC02, TC03) were approved immediately and the dashboard stayed SAFE with a valid safe sequence. The unsafe request in TC07 was queued although 5 seats were free, and it was approved automatically after the cancellation in TC09. The state of the hall during this scenario is summarised below.

Table 5.2: State of the hall during the scenario TC01 – TC09 (order: Computer / IT / ENTC)

Step	Operation	Available	Allocation	Need	Status / safe sequence
0	Start-up	10	0 / 0 / 0	7 / 5 / 6	SAFE: Computer, IT, ENTC
1	Computer books 2	8	2 / 0 / 0	5 / 5 / 6	SAFE
2	IT books 3	5	2 / 3 / 0	5 / 2 / 6	SAFE: Computer, IT, ENTC
3	ENTC requests 4	5	2 / 3 / 0	5 / 2 / 6	Refused (unsafe); queued
4	IT cancels 1	6	2 / 2 / 0	5 / 3 / 6	Queue rescanned
5	ENTC 4 approved	2	2 / 2 / 4	5 / 3 / 2	SAFE: ENTC, Computer, IT

Boundary inputs. A group booking its whole maximum (TC10) was accepted; a booking equal to the number of free seats is accepted when it is safe, while a request larger than the free seats (TC13) is queued. A cancellation of all seats held (TC15) releases the hall completely and triggers the processing of the queue.

Invalid inputs. Wrong group numbers, zero or excessive seat counts, over-cancellation, cancellation by a group with no seats, wrong menu choices and invalid start-up values were rejected with clear messages (TC04–TC06, TC11, TC12, TC16–TC18). The only unhandled case was non-numeric input (TC19).

5.3 Result Analysis and Observations

Trace of the unsafe request (TC07). The table shows why the Banker's Algorithm refuses ENTC Dept's request for 4 seats, and why the request is accepted after IT Dept releases a seat.

Table 5.3: Safety check before and after the cancellation (order: Computer / IT / ENTC)

Situation	Available after trial	Need after trial	Groups that can finish	Result
ENTC requests 4 (before cancellation)	1	5 / 2 / 2	None (every need > 1)	UNSAFE – queued

Situation	Available after trial	Need after trial	Groups that can finish	Result
IT cancels 1, ENTC 4 retried	2	5 / 3 / 2	ENTC ($2 \le 2$) → Work 6; Computer ($5 \le 6$) → Work 8; IT ($3 \le 8$)	SAFE – approved

Comparison with plain first-come allocation. A system that only checks whether enough seats are free would have granted ENTC's 4 seats, leaving 1 free seat while Computer, IT and ENTC still need 5, 2 and 2 seats. No group could complete, so none would release seats – a deadlock. SmartSem's check prevented this state.

Table 5.4: First-come allocation versus SmartSem (Banker's Algorithm)

Aspect	First-come allocation	SmartSem (Banker's Algorithm)
Decision rule	Grant if seats ≤ free seats	Grant if seats ≤ free seats AND the resulting state is safe
ENTC requests 4 (5 free)	Granted; 1 seat left, no group can finish	Queued; state stays safe
Risk of stuck groups	Possible (hold and wait)	Avoided
Queued requests	None – requests are simply refused	FIFO list, re-examined after every cancellation
Cost per request	O(1)	$O(n^2)$ for the safety check

Observations.

- Every booking that was approved left the system SAFE; a safe sequence was displayed each time, and the sequence changes with the allocation (Computer, IT, ENTC before TC09 and ENTC, Computer, IT after it).
- Unsafe requests were never granted, but they were not lost: they were approved automatically as soon as a cancellation made them safe (TC09, TC15).
- FIFO fairness works: TC14 shows that a small request (1 seat) does not overtake an older, larger request even though it would fit.
- Because only safe requests are granted and a release keeps a safe state safe, the UNSAFE branch of the dashboard is a safeguard that is not reached in normal use.
- Every invalid input that was tried was rejected with a clear message, except non-numeric input (TC19).

5.4 Limitations

- **Non-numeric input** (TC19) is not handled: Scanner.nextInt() throws InputMismatchException and the program ends. A try–catch or hasNextInt() check is needed.
- Only **one resource type** (seats in one hall for one event) is modelled; dates, time slots and

several halls are not.

- Seats are only counted; **individual seat numbers** are not assigned.
- In processWaitingList() every waiting request is tried in FIFO order, but a later small request can be approved while an earlier large request still has to wait, so a very large request could wait for a long time.
- All data are kept **in memory**; closing the program loses the bookings.
- The dashboard uses tab characters, so columns can look uneven when group names have very different lengths.
- The program assumes that groups declare a correct maximum in advance and release seats in finite time; it has no authentication or multi-user access.

CONCLUSION

SmartSem implements the Banker's Algorithm for a realistic task – reserving seats in a college seminar hall. The program accepts the hall capacity and the maximum needs of the groups, grants a booking only when the resulting state is safe, queues all other requests in a FIFO waiting list, releases seats through cancellation and re-examines the queue automatically. A dashboard shows the allocation, the SAFE/UNSAFE status and a safe sequence at any time.

All eight objectives of Section 1.4 were achieved. Of the 19 test cases, 18 passed and the failing case (non-numeric input) is documented as a limitation. The project gave practical understanding of safe and unsafe states, the need calculation, safe sequences, deadlock avoidance versus plain allocation, and fairness in waiting queues, and showed that an operating-system concept can be applied directly to a shared resource in everyday campus life.

FUTURE SCOPE

- Add input validation with try–catch / hasNextInt() so that wrong data types never stop the program.
- Extend the model to several halls or time slots, i.e. several resource types, using the vector form of the Banker's Algorithm.
- Assign named or numbered seats and generate printable booking slips.
- Store bookings in a file or database so that they persist between runs.
- Provide a graphical or web interface with a live dashboard and a colour-coded safe/unsafe indicator.

- Add priorities (for example for the annual function) and aging so that large requests in the waiting list cannot starve.
- Support several coordinators using the system at once through a thread-safe implementation and notifications when a waiting request is approved.

REFERENCES

[1] A. Silberschatz, P. B. Galvin and G. Gagne, Operating System Concepts, 10th ed. Hoboken, NJ: Wiley, 2018 (chapter on Deadlocks: deadlock avoidance, safe state and the Banker's Algorithm).

[2] A. S. Tanenbaum and H. Bos, Modern Operating Systems, 4th ed. Harlow: Pearson, 2015 (chapter on Deadlocks).

[3] D. M. Dhamdhere, Operating Systems: A Concept-Based Approach, 3rd ed. New Delhi: McGraw-Hill Education, 2012.

[4] E. W. Dijkstra, "Een algorithme ter voorkoming van de dodelijke omarming," EWD108, 1965 (original description of the banker's algorithm).

[5] A. N. Habermann, "Prevention of system deadlocks," Communications of the ACM, vol. 12, no. 7, pp. 373–377, 1969.

[6] Oracle, Java Platform, Standard Edition API Specification: java.util.Scanner, java.util.Queue and java.util.LinkedList, docs.oracle.com.

[7] Project source code: SeminarHallReservation.java (given in Section 4.3(f) of this report).